TYPESCRIPT / TYPE-CHALLENGES

IsAny はなぜ any だけを見抜けるのか

西 悠太　/ 株式会社ダイニー

自己紹介

西 悠太 (Nishi Yuta)

V8 Contributor

Platform Engineer at Dinii Inc.

TSKaigi Staff

TypeScriptが好きです。
型パズルも好きです。

@riya-amemiya



お題は IsAny

type-challengesに、anyだけをtrueにするIsAnyを作ろうという問題があります。

```
type IsAny<T> = /* ??? */

type A = IsAny<any>     // true
type B = IsAny<unknown>  // false
type C = IsAny<never>    // false
type D = IsAny<string>   // false
```

一見簡単そうですが、
実際解き始めるとうまくいきません。

まずextends の意味をおさらい

extendsはよく説明の時にif文のように説明されますが、正確には左辺は右辺の部分集合かどうかがイメージとしては近いです。 厳密には部分集合も説明としては違いますが、ここではイメージの話なのでextendsは部分集合と説明します。

```
type A = 1 extends 1 ? true : false  // true
type B = 1 extends 2 ? true : false  // false
```

じゃあanyにextendsしてあげると良さそうですね。

試み① T extends any

```
type IsAny<T> = T extends any ? true : false

type A = IsAny<any>     // true
type D = IsAny<string>  // true  ← string でも true
```

どんな型でも any には代入できるので、any 以外まで true になってしまいます。

試み② unknown extends T

```
type IsAny<T> = unknown extends T ? true : false

type A = IsAny<any>     // true
type B = IsAny<unknown>  // true ← unknown まで true
type D = IsAny<string>   // false
```

unknown を受け取れるのは any と unknown だけ。
惜しいところまで来ますが、unknown を除外できません。
IsAnyはanyだけをtrueにしたいので、これもダメです。

ですがanyの特性をうまく使うと
とてもスマートに書けます。

答えは、この1行

```
type IsAny<T> = 0 extends 1 & T ? true : false
// 0と1の部分は絶対に重ならない2つのリテラル型ならなんでもいいです
```

交差型について考えてみる

交差型はAとBの両方を満たす型を作れる演算子です。
基本的には狭い方を優先します。
先ほどの例を簡単に表現すると

```
type IsAny<T> = 0 extends 1 & T ? true : false
```

は「Tは1と交差可能で、かつ0の部分集合になる型である」と言い換えられるのではないでしょうか。
1はnumberより狭い型なので、普通に考えるとこれを満たす型はないはずです。

分解① 1 & T

```
type T1 = 1 & number   // 1
type T2 = 1 & string   // never
type T3 = 1 & unknown  // 1
type T4 = 1 & never    // never
type T5 = 1 & any      // any
```

anyだけは全ての型と交差可能で、その交差の結果はanyになります。

分解② 0 extends 1

```
type R1 = 0 extends 1     ? true : false  // false
type R2 = 0 extends never ? true : false  // false
type R3 = 0 extends any   ? true : false  // true
```

0 は 1 にも never にも代入できないので、`0 extends 1 & T` は本来「絶対に false」の式です
それを true にできる唯一の抜け道が any です。

評価を追う

```
IsAny<string>   0 extends 1 & string   → 0 extends never  → false
IsAny<number>   0 extends 1 & number   → 0 extends 1      → false
IsAny<unknown>  0 extends 1 & unknown  → 0 extends 1      → false
IsAny<any>      0 extends 1 & any      → 0 extends any    → true
```

前述の2つの性質を同時に持つ型は any しかありません

any は、上でも下でもある

unknown

何でも受け取れるが、他の型には入れない

never

どの型にも入れるが、何も受け取れない

any

両方できる。型の階層の外にいる

any は「型チェックをやめる」ための型なので、代入の階層に縛られません。
IsAny は、この階層の外にいる性質そのものを検出しています。

おまけ① 条件型は両方の分岐を返す

```
type X = any extends string ? 1 : 2   // 1 | 2
```

判定される側に any を置くと、true とも false とも決められないので、
両方の枝のユニオンが返ってきます。

おまけ② any でも代入できない時がある

```
declare const a: any

const s: string = a  // OK
const n: never = a   // Error: Type 'any' is not assignable to type 'never'
```

何にでも代入できる any が、唯一 never にだけは入れません。

まとめ

- ✓ IsAny は `0 extends 1 & T` の1行で書ける
- ✓ どんな型とも交差できるのは any だけ
- ✓ 「絶対に false」の代入を true にできるのも any だけ
- ✓ any は上にも下にも振る舞う、型の階層の外の型

参考資料

- GitHub: **type-challenges/type-challenges**
- TypeScript Handbook: **Everyday Types — any**

ご清聴ありがとうございました

0 extends 1 & T。

この1行に、any のすべてが詰まっている。